

CELEBAL EXCELLENCE INTERNSHIP
PROGRAM

DATA ENGINEERING

WEEK - 7 ASSIGNMENT REPORT

DATABRICKS

SUBMITTED BY:-

ANUSHKA SHARMA

(JKLU)

1. OBJECTIVE

The objective of this assignment was to implement incremental data processing using Delta Lake. The pipeline demonstrates how raw data is ingested, cleaned, updated using incremental records, and transformed into analytics-ready tables using Apache Spark and Delta Lake.

The main focus of this implementation is the Delta Lake MERGE operation, which enables efficient updating of existing records and inserting new records without rewriting the complete dataset.

2. TECHNOLOGIES USED

- Databricks
- Apache Spark
- PySpark
- Delta Lake

3. DATA PIPELINE ARCHITECTURE

The implementation follows the Medallion Architecture approach:

Raw CSV Dataset → Bronze Layer → Silver Layer → Incremental MERGE Processing
→ Gold Layer

4. BRONZE LAYER - Raw Data Ingestion

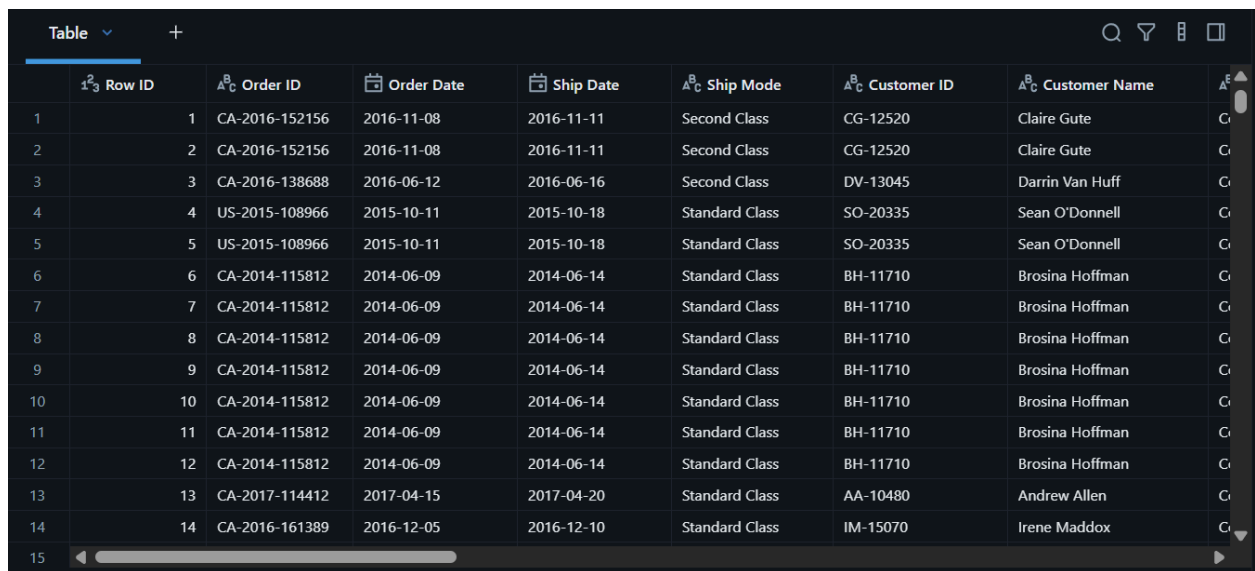
The Sample Superstore dataset was loaded from the Unity Catalog Volume into a Spark DataFrame.

The raw dataset was stored as a Delta table named: **“bronze”**

The Bronze layer stores the data in its original form and acts as the initial storage layer before performing any transformations.

Operations performed:

- Loaded CSV file using Spark DataFrame API
- Enabled header reading and schema inference
- Stored the dataset in Delta format



Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name
1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gute
3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell
5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell
6	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman
7	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman
8	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman
9	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman
10	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman
11	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman
12	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710	Brosina Hoffman
13	CA-2017-114412	2017-04-15	2017-04-20	Standard Class	AA-10480	Andrew Allen
14	CA-2016-161389	2016-12-05	2016-12-10	Standard Class	IM-15070	Irene Maddox

5. SILVER LAYER - Data Cleaning

The Bronze data was processed to create a cleaned Silver layer.

The following cleaning operations were performed:

Null Value Checking

All columns were checked for missing values.

```

▶ 01:47 PM (1s) 13

from pyspark.sql.functions import col, when, count

null_counts = silver_df.select([
    count.when(col(c).isNull(), c).alias(c)
    for c in silver_df.columns
])

display(null_counts)

> See performance (1) Optimize

> null_counts: pyspark.sql.connect.dataframe.DataFrame = [Row ID: long, Order ID: long ... 19 more fields]

```

1	2 Row ID	3 Order ID	3 Order Date	3 Ship Date	3 Ship Mode	3 Customer ID	3 Customer Name	3 S
1	0	0	0	0	0	0	0	

Observation:

- No null values were found in the dataset.
- Therefore, no additional null handling was required.

Duplicate Removal

Duplicate records were checked and removed using: **dropDuplicates()**

```

▶ 01:47 PM (1s) 15

rows_before = silver_df.count()
silver_df = silver_df.dropDuplicates()
rows_after = silver_df.count()

print(f"Rows Before: {rows_before}")
print(f"Rows After : {rows_after}")
print(f"Duplicates Removed: {rows_before - rows_after}")

> See performance (2)

> silver_df: pyspark.sql.connect.dataframe.DataFrame = [Row ID: integer, Order ID: string ... 19 more fields]
Rows Before: 9994
Rows After : 9994
Duplicates Removed: 0

```


7. Delta Lake MERGE Operation

The main objective of this assignment was achieved using the Delta Lake MERGE operation.

The MERGE operation performs an upsert process:

- If a matching record exists, the existing record is updated.
- If a matching record does not exist, a new record is inserted.

The merge condition was based on “**Row ID**”

The implementation used “**whenMatchedUpdateAll()**” to update existing records.

And “**whenNotMatchedInsertAll()**” to insert new records.

This allows incremental changes to be processed efficiently without replacing the complete dataset.

MERGE Result

Before MERGE: Silver Table Records: 9994, Incremental Records: 20



```
▶ ▼ ✓ 01:47 PM (1s) 22

print("Bronze Rows :", spark.table("bronze").count())
print("Silver Rows :", spark.table("silver").count())
print("Incremental Rows :", spark.table("incremental").count())

> See performance \(3\)

Bronze Rows : 9994
Silver Rows : 9994
Incremental Rows : 20
```

After MERGE: Final Silver Table Records: 10004



```
▶ ✓ 01:47 PM (<1s)

print("Rows after MERGE:", spark.table("silver").count())

> See performance \(1\)

Rows after MERGE: 10004
```

Based on the final row count, the MERGE operation successfully updated existing records and inserted new records into the Silver Delta table without creating duplicate Row IDs.

8. VALIDATION

After performing the MERGE operation, validation checks were performed.

The following validations were completed:

Row Count Validation- Final record count was checked to confirm successful processing.

Result: Final Row Count: 10004

Duplicate Validation- Duplicate Row IDs were checked after MERGE.

Result: Duplicate Row IDs: 0

This confirms that the MERGE operation maintained data consistency.

```
▶ 01:47 PM (<1s) 28
print("Final Row Count:", spark.table("silver").count())
> See performance \(1\)
Final Row Count: 10004

▶ 01:47 PM (1s) 29
duplicate_count=(spark.table("silver").groupBy("Row ID").count().filter("count > 1").count())
print("Number of Duplicate Row IDs:", duplicate_count)
> See performance \(1\)
Number of Duplicate Row IDs: 0
```

9. GOLD LAYER - Analytical Tables

After completing incremental processing, business-ready tables were created from the updated Silver layer.

Fact Table

The fact table contains transactional information such as:

- Order details
- Sales
- Quantity
- Discount
- Profit

Created table: “**fact_table**”

Customer Dimension

The customer dimension stores customer-related attributes:

- Customer ID

- Customer Name
- Segment
- Location details

Created table: “**dim_customer**”

Product Dimension

The product dimension stores product-related attributes:

- Product ID
- Product Name
- Category
- Sub-category

Created table: “**dim_product**”

▶ ▼ ✓ 01:47 PM (1s)

```
display(spark.sql("SHOW TABLES"))
```

> [See performance \(2\)](#)

	database	tableName	isTemporary
1	`week7-assignme...	bronze	false
2	`week7-assignme...	dim_customer	false
3	`week7-assignme...	dim_product	false
4	`week7-assignme...	fact_table	false
5	`week7-assignme...	incremental	false
6	`week7-assignme...	silver	false

↓ 6 rows | 0.585s runtime

10.CONCLUSION

This assignment successfully demonstrated incremental data processing using Delta Lake in Databricks. The Medallion Architecture was used to organize data into Bronze, Silver, and Gold layers. Delta Lake's MERGE operation efficiently handled updates and inserts from an incremental dataset while maintaining data consistency. Finally, analytical fact and dimension tables were created to support downstream reporting and analysis.